Toronto Metropolitan University

Department of Electrical, Computer & Biomedical Engineering
Faculty of Engineering & Architectural Science

Program: Computer Engineering

Course Number	COE 892
Course Title	Distributed Cloud Computing
Semester/Year	Winter 2026
Instructor	Muhammad Jaseemuddin

Report Title	Lab 2

Section No.	09
Submission Date	February 25, 2026
Due Date	February 27, 2026

Name	Student ID	Signature*
Krish Patel	xxxx91722	K.P

(Note: remove the first 4 digits from your student ID)

*By signing above you attest that you have contributed to this submission and confirm that all work you have contributed to this submission is your own work. Any suspicion of copying or plagiarism in this work will result in an investigation of Academic Misconduct and may result in a "0" on the work, an "F" in the course, or possibly more severe penalties, as well as a Disciplinary Notice on your academic record under the Student Code of Academic Conduct, which can be found online at: https://www.torontomu.ca/content/dam/senate/policies/pol60.pdf

COE892 Lab 2

Introduction

This lab replaces the file driven control loop from Lab 1 with a gRPC based client server design. Instead of each rover reading everything locally, a single ground control server (GCS) becomes the source truth for the shared 2D map, rover command streams, and mine metadata. Each rover is a gRPC client that connects to the server, pulls what it needs, executes its route, and reports its outcome back to the GCS.

The main goal is not raw performance. The focus is correctness and clarity of distributed communication: structured messages, a server streaming RPC for commands, and a clean end to end interaction that can be demonstrated reliably.

Problem definition and requirements

The system consists of one gRPC server (ground control) and 10 clients (rovers). Each rover must support interactions with the server: retrieving the map, receiving a stream of commands, requesting a mine serial number, reporting whether it completed successfully or not, and sending a mine PIN with the server.

One detail that affects the overall design is the independence assumption: rovers do not coordinate with each other. If two rovers reach the same mine, each rover must still disarm it as if it were not disarmed before. That rule means the server should not “lock out” a mine after one rover completes it.

Implementation details

The implementation is split into a Protocol Buffer definition and two Python programs: a ground control server (GCS.py) and the rover client (rover.py). The .proto file defines a single service, Ground Control, and five RPC methods that map directly to the required behaviors.

Most of the communication is unary request response, but commands are delivered using a server streaming RPC. That choice is intentional: it makes command delivery feel like a live feed rather than a single bulk response, and it demonstrates a core gRPC pattern that is hard to show with only unary RPCs.

The project artifacts used in the lab run are:

- rover.proto for the service and message schema
- generated stubs: rover_pb2.py and rover_pb2_grpc.py
- GCS.py as the server
- rover.py as the client runner (single rover and all rover modes)
- input/map.txt, input/mines.txt, and input/commands/rover_<id>.txt

gRPC methods and message flow

The RPCs line up one to one with the lab statement:

GetMap
The rover requests the map at startup. The server returns the map as a list of strings (rows) and basic metadata (width, height, start position).

StreamCommands
The rover requests its command stream using its rover id. The server streams chunks of the command string until an end marker is sent.

GetMineSerial
When a rover is on a mine cell and attempts to disarm, it requests the serial number for that location.

ShareMinePin
After receiving the serial, the rover computes a PIN deterministically and sends it to the server. The server returns an acknowledgement indicating whether the PIN is accepted.

ReportExecution
At the end of execution, the rover sends a summary report that includes success or explosion status, how many commands were executed, and the final position.

Rover execution logic

Each rover run follows the same sequence: connect, fetch the map, fetch commands, execute commands in order, and report the outcome. The rover stores the map locally and maintains its own state, including direction, position, and whether it is currently standing on a mine that requires disarming.

Commands are interpreted sequentially. Turn commands update direction, move commands attempt to advance one cell (while respecting blocked cells), and disarm commands trigger mine handling. The mine handling portion is the main "multi RPC" part of the rover's logic: it calls GetMineSerial, computes a PIN, calls ShareMinePin, then continues or marks itself exploded depending on the server's acknowledgement.

Even though the system can run multiple rovers concurrently for testing, the rover's internal logic remains sequential, which matches the lab requirement that threading is not necessary for the rover's command execution.

Execution modes

The lab only requires a rover program that runs one rover given an id. That requirement is met directly with the single rover CLI mode. For convenience and for clearer demonstrations, the rover client also supports running all 10 rovers in a single command in either sequential or threaded concurrency mode.

Sequential mode:
python rover.py 1 --host localhost:50051

All rovers sequential:
python rover.py --all --mode sequential --host localhost:50051

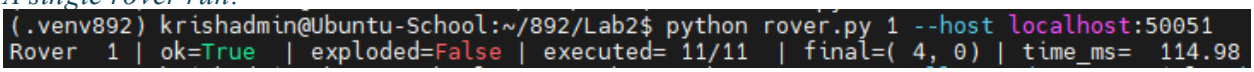
All rovers threaded concurrency:
python rover.py --all --mode concurrent --workers 10 --host localhost:50051

In both all rover modes, each rover still behaves like an independent gRPC client. The "concurrent" mode simply dispatches multiple rover runs using a thread pool so their gRPC activity overlaps in time.

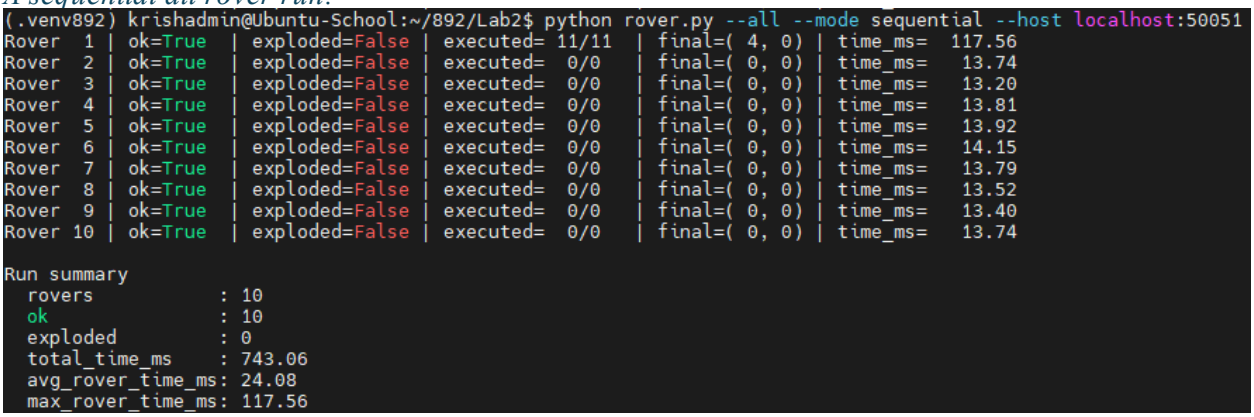
Results

Timing is measured inside the rover client using time.perf_counter(). Each rover prints a single line containing its outcome and elapsed time. When running with --all, a short run summary is printed after the per rover lines.

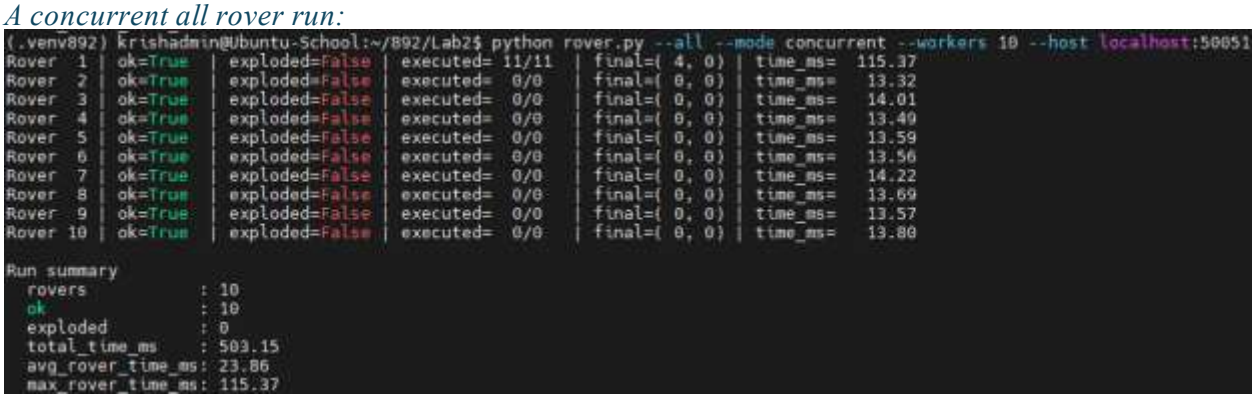
A single rover run:



A sequential all rover run:



A concurrent all rover run:



Server Side Output:

```
(.venv892) krishadmin@Ubuntu-School:~/892/Lab2$ python GCS.py --host 0.0.0.0:50051
2026-02-09 04:09:37,834 INFO Ground Control running on 0.0.0.0:50051
2026-02-09 04:09:56,087 INFO Rover 1 report: success=True exploded=False executed=11/11 final=(4,0) msg=ok
2026-02-09 04:09:59,929 INFO Rover 1 report: success=True exploded=False executed=11/11 final=(4,0) msg=ok
2026-02-09 04:09:59,993 INFO Rover 2 report: success=True exploded=False executed=0/0 final=(0,0) msg=ok
2026-02-09 04:10:00,056 INFO Rover 3 report: success=True exploded=False executed=0/0 final=(0,0) msg=ok
2026-02-09 04:10:00,120 INFO Rover 4 report: success=True exploded=False executed=0/0 final=(0,0) msg=ok
2026-02-09 04:10:00,183 INFO Rover 5 report: success=True exploded=False executed=0/0 final=(0,0) msg=ok
2026-02-09 04:10:00,247 INFO Rover 6 report: success=True exploded=False executed=0/0 final=(0,0) msg=ok
2026-02-09 04:10:00,311 INFO Rover 7 report: success=True exploded=False executed=0/0 final=(0,0) msg=ok
2026-02-09 04:10:00,375 INFO Rover 8 report: success=True exploded=False executed=0/0 final=(0,0) msg=ok
2026-02-09 04:10:00,439 INFO Rover 9 report: success=True exploded=False executed=0/0 final=(0,0) msg=ok
2026-02-09 04:10:00,502 INFO Rover 10 report: success=True exploded=False executed=0/0 final=(0,0) msg=ok
2026-02-09 04:10:21,327 INFO Rover 2 report: success=True exploded=False executed=0/0 final=(0,0) msg=ok
2026-02-09 04:10:21,378 INFO Rover 3 report: success=True exploded=False executed=0/0 final=(0,0) msg=ok
2026-02-09 04:10:21,383 INFO Rover 1 report: success=True exploded=False executed=11/11 final=(4,0) msg=ok
2026-02-09 04:10:21,429 INFO Rover 4 report: success=True exploded=False executed=0/0 final=(0,0) msg=ok
2026-02-09 04:10:21,479 INFO Rover 5 report: success=True exploded=False executed=0/0 final=(0,0) msg=ok
2026-02-09 04:10:21,529 INFO Rover 6 report: success=True exploded=False executed=0/0 final=(0,0) msg=ok
2026-02-09 04:10:21,579 INFO Rover 7 report: success=True exploded=False executed=0/0 final=(0,0) msg=ok
2026-02-09 04:10:21,629 INFO Rover 8 report: success=True exploded=False executed=0/0 final=(0,0) msg=ok
2026-02-09 04:10:21,680 INFO Rover 9 report: success=True exploded=False executed=0/0 final=(0,0) msg=ok
2026-02-09 04:10:21,729 INFO Rover 10 report: success=True exploded=False executed=0/0 final=(0,0) msg=ok
2026-02-09 04:12:15,723 INFO Rover 1 report: success=True exploded=False executed=11/11 final=(4,0) msg=ok
2026-02-09 04:12:19,611 INFO Rover 1 report: success=True exploded=False executed=11/11 final=(4,0) msg=ok
2026-02-09 04:12:19,675 INFO Rover 2 report: success=True exploded=False executed=0/0 final=(0,0) msg=ok
2026-02-09 04:12:19,739 INFO Rover 3 report: success=True exploded=False executed=0/0 final=(0,0) msg=ok
2026-02-09 04:12:19,803 INFO Rover 4 report: success=True exploded=False executed=0/0 final=(0,0) msg=ok
2026-02-09 04:12:19,867 INFO Rover 5 report: success=True exploded=False executed=0/0 final=(0,0) msg=ok
2026-02-09 04:12:19,931 INFO Rover 6 report: success=True exploded=False executed=0/0 final=(0,0) msg=ok
2026-02-09 04:12:19,995 INFO Rover 7 report: success=True exploded=False executed=0/0 final=(0,0) msg=ok
2026-02-09 04:12:20,059 INFO Rover 8 report: success=True exploded=False executed=0/0 final=(0,0) msg=ok
2026-02-09 04:12:20,123 INFO Rover 9 report: success=True exploded=False executed=0/0 final=(0,0) msg=ok
2026-02-09 04:12:20,187 INFO Rover 10 report: success=True exploded=False executed=0/0 final=(0,0) msg=ok
2026-02-09 04:14:47,839 INFO Rover 1 report: success=True exploded=False executed=11/11 final=(4,0) msg=ok
2026-02-09 04:14:52,457 INFO Rover 1 report: success=True exploded=False executed=11/11 final=(4,0) msg=ok
2026-02-09 04:14:52,521 INFO Rover 2 report: success=True exploded=False executed=0/0 final=(0,0) msg=ok
2026-02-09 04:14:52,585 INFO Rover 3 report: success=True exploded=False executed=0/0 final=(0,0) msg=ok
2026-02-09 04:14:52,649 INFO Rover 4 report: success=True exploded=False executed=0/0 final=(0,0) msg=ok
2026-02-09 04:14:52,713 INFO Rover 5 report: success=True exploded=False executed=0/0 final=(0,0) msg=ok
2026-02-09 04:14:52,777 INFO Rover 6 report: success=True exploded=False executed=0/0 final=(0,0) msg=ok
2026-02-09 04:14:52,841 INFO Rover 7 report: success=True exploded=False executed=0/0 final=(0,0) msg=ok
2026-02-09 04:14:52,905 INFO Rover 8 report: success=True exploded=False executed=0/0 final=(0,0) msg=ok
2026-02-09 04:14:52,969 INFO Rover 9 report: success=True exploded=False executed=0/0 final=(0,0) msg=ok
2026-02-09 04:14:53,033 INFO Rover 10 report: success=True exploded=False executed=0/0 final=(0,0) msg=ok
2026-02-09 04:14:58,743 INFO Rover 2 report: success=True exploded=False executed=0/0 final=(0,0) msg=ok
2026-02-09 04:14:58,794 INFO Rover 3 report: success=True exploded=False executed=0/0 final=(0,0) msg=ok
2026-02-09 04:14:58,794 INFO Rover 1 report: success=True exploded=False executed=11/11 final=(4,0) msg=ok
2026-02-09 04:14:58,844 INFO Rover 4 report: success=True exploded=False executed=0/0 final=(0,0) msg=ok
2026-02-09 04:14:58,894 INFO Rover 5 report: success=True exploded=False executed=0/0 final=(0,0) msg=ok
2026-02-09 04:14:58,944 INFO Rover 6 report: success=True exploded=False executed=0/0 final=(0,0) msg=ok
2026-02-09 04:14:58,995 INFO Rover 7 report: success=True exploded=False executed=0/0 final=(0,0) msg=ok
2026-02-09 04:14:59,044 INFO Rover 8 report: success=True exploded=False executed=0/0 final=(0,0) msg=ok
2026-02-09 04:14:59,094 INFO Rover 9 report: success=True exploded=False executed=0/0 final=(0,0) msg=ok
2026-02-09 04:14:59,145 INFO Rover 10 report: success=True exploded=False executed=0/0 final=(0,0) msg=ok
```

Notes:

The executed=0/0 results for rovers 2 to 10 indicate that those rover ids received an empty command stream. With the current server behavior, that occurs when input/commands/rover_<id>.txt is missing or empty. In that case, the rover has nothing to execute, and it correctly reports that it completed all commands it was given. The short non zero time is mostly gRPC overhead, plus any configured small sleeps in streaming and execution loops.

This is still a correct run, but it is not a very informative benchmark. To produce more meaningful comparisons between sequential and concurrent multi rover runs, each rover should have a non empty command file so command execution time dominates the fixed RPC overhead.

Discussion

From a functional perspective, the end-to-end gRPC interaction is behaving as intended. The server is reachable, the rover can fetch the map and stream commands, and the server reliably receives completion reports. The “multi step” mine interaction is also supported by the service design, since it requires the rover to perform multiple RPC calls during a single run.

The test timing results should be interpreted carefully. With short command streams or empty command streams, the runtime is dominated by constant overhead (channel creation, RPC setup, small intentional sleeps), which makes the total runtime sensitive to scheduling noise. That is especially visible when comparing sequential vs threaded concurrency. In other words, concurrency here is more about validating that the server handles multiple clients cleanly than about proving a large speedup.

The independence requirement is satisfied by design because mines are not globally marked as “already disarmed” in a way that prevents other rovers from interacting with them. Each rover is allowed to request serials and submit PINs based solely on its own path and current position.

Conclusion

This lab implemented a gRPC based rover to ground control simulation that matches the five required behaviors: map retrieval, command streaming, mine serial retrieval, mine PIN sharing, and final execution reporting. The .proto schema cleanly describes the service interface, the server loads the same style of input data used in Lab 1, and each rover client follows a consistent fetch execute report workflow.

The system is demonstrable using either single rover runs or all rover runs in sequential and threaded concurrency modes. Timing instrumentation is included to make behavior observable and to support reporting, with the understanding that meaningful performance comparisons require nontrivial command streams for all rovers.